Azure AI & Microsoft Foundry

Comprehensive Compilation of Exam Scenarios, Formats, and Explanations

Drag and Drop Code Completion

Scenario 1: Forcing Tool Execution in Ticket Triage

You have a Microsoft Foundry project that contains a deployed ticket-triage agent. You discover that sometimes the agent responds without calling any tools, even when a tool is required. You need to ensure that the agent calls a tool during execution. How should you complete the Python code payload?

Target Key Box	Target Value Box
"tool_choice"	**"required"**

```
run_payload = {
    "assistant_id": agent_id,
    "tool_choice": "required",
    "metadata": {
        "scenario": "ticket-triage"
    }
}
```

Explanation: In the OpenAI Assistants / Azure AI Agent Service payload, setting `"tool_choice": "required"` explicitly forces the agent to choose and execute at least one tool before returning a text response, overriding the default `"auto"` behavior which can selectively skip tool paths.

Dropdown Selection

Scenario 2: CI/CD Workflow Evaluation & Branch Protection

You have a Microsoft Foundry project that contains an agent. You use a GitHub Actions workflow for CI/CD. You need to configure the workflow to automatically evaluate the agent when a pull request (PR) is created and prevent branches from merging if the evaluation results do NOT meet the defined thresholds. How should you configure the workflow?

Configuration Property	Selected Option (Correct Answer)
Authentication method	**An Azure Login action that uses OpenID Connect (OIDC)**
If the evaluation results are NOT met, configure the workflow to	**Fail**

Explanation: OIDC is the security standard for credential-less cloud authentication inside GitHub Actions. To block branches from merging, the workflow run itself must explicitly **Fail** to toggle the repository's status checks red and block PR merges.

Multiple Choice

Scenario 3: Real-Time Audio Chat Turn-Taking Dynamic

You are creating an agent workflow in a Microsoft Foundry project to support natural voice interactions. The agent must receive continuous audio input, convert the input into text for reasoning, and then return spoken responses to a user. The workflow must support turn-taking dynamics where the agent begins to generate speech output before the user finishes speaking, operating with low latency. What should you do?

A. Use speech translation to convert the audio into another language and return the translated text.

B. Use real-time speech to text for incoming audio and text to speech for agent responses.

C. Use an embeddings model to encode the audio, and then decode the audio into text and speech.

D. Use batch transcription to convert the audio input and return text responses from the agent.

Explanation: To implement low-latency continuous streaming conversations, full-duplex streaming via real-time speech-to-text (STT) combined with real-time text-to-speech (TTS) is necessary. Batch pipelines add high latency, rendering them completely unusable here.

Multiple Choice

Scenario 4: Live Call Transcription Monitor

You are building a speech processing solution in Microsoft Foundry for a customer support platform. The platform will transcribe live phone calls so that supervisors can view transcripts and detect issues while the calls are in progress. The call audio arrives as a continuous stream from the telephony system. You need to ensure that transcripts appear within only a few seconds of the audio stream. What should you do?

A. Use speech translation to generate the transcripts into multiple languages.

B. Use text to speech by using a custom neural voice.

C. Run a batch transcription job on recorded audio files.

D. Use real-time speech to text to process streaming audio input.

Explanation: Live, continuous streams originating from active infrastructure must use real-time streaming endpoints to surface text blocks inside a few seconds. Batch parsing cannot connect to a live pipeline stream.

Multiple Choice

Scenario 5: Fine-tuned Speech Model REST API Authorization Error

You have a Microsoft Foundry project that contains an agent. The agent uses Azure Speech in Foundry Tools. You fine-tune a baseline speech to text model for the en-US locale and publish the model. The agent calls the Speech to text REST API and returns an error message indicating that the project ID is invalid. You need to set the project property to the correct ID. To what should you set the project property?

A. the project URL

B. the custom speech project ID

C. the project ID

D. the custom speech endpoint URL

Explanation: Fine-tuning speech assets occurs inside specialized Custom Speech studios. The core Speech service REST endpoint requires the targeted **custom speech project ID** context parameter to route the calls to the custom fine-tuned weights rather than standard models.

Multiple Choice

Scenario 6: Cost Observability and Analysis

You have a Microsoft Foundry project that contains a high-traffic agent. After a recent update, operational costs increase significantly. Monitoring confirms that the volume of user traffic remains unchanged. You suspect that changes to the request or response characteristics are causing the increase. You need to identify whether additional costs are driven by model input size, model output size, or expanded tool usage. Which observability capability should you use?

A. evaluation metrics

B. run success rate

C. latency

D. token usage

Explanation: Operational charges in generative LLM workflows map directly to **token usage**. Tracking prompt (input) tokens, completion (output) tokens, and total context window inflation provides the literal metrics driving costs.

Multiple Choice

Scenario 7: Least Privilege Inference Authorization (RBAC)

You have a Microsoft Foundry project that contains a model deployment. You have an application that calls the deployment by using the Azure OpenAI v1 API and DefaultAzureCredential. The developers receive HTTP 403 errors when they send inference requests, even after running 'az login'. You need to ensure the developers can perform model inference following the principle of least privilege. Which role-based access control (RBAC) role should you assign?

A. Contributor

B. Cognitive Services Data Reader

C. Cognitive Services OpenAI User

D. Cognitive Services User

Explanation: The **Cognitive Services OpenAI User** role isolates permissions exclusively to the data plane, enabling deployment inference (completions/chat) without opening wide administrative control-plane scopes (like Contributor).

Multiple Choice

Scenario 8: Deterministic Multi-Agent Orchestration

You have a Microsoft Foundry project that contains three agents: TriageAgent (classifies requests), PolicyAgent (searches content), and ActionAgent (calls APIs). You need to orchestrate the agents to ensure customer requests meet requirements: support a deterministic, step-based process using conditional branching, shared state across agents, and optionally trigger a ticket action based on triage. The solution must minimize development effort. What should you include?

A. a workflow

B. a multi-agent group chat session

C. separate agent runs coordinated in the application code

D. threads and runs without a workflow

Explanation: Visual or prompt **workflows** in Azure Foundry natively support step-based deterministic routing, conditional execution blocks, and unified shared state boundaries out-of-the-box, removing the need to write heavy boilerplate app orchestration logic.

Multiple Choice

Scenario 9: Multi-Turn Parallel Vector Index Ingestion (RAG)

You have a chat app in a Microsoft Foundry project and an Azure AI Search vectorized index. You need to connect to the index to meet the requirements: complex questions must retrieve information from multiple chunks, multi-turn conversations must influence retrieval planning, and retrievals must run in parallel to reduce latency. Which retrieval approach should you use?

A. iterative retrieval

B. classic Retrieval Augmented Generation (RAG)

C. agentic Retrieval Augmented Generation (RAG)

D. chain of thought

Explanation: **Agentic RAG** introduces an LLM agent reasoning step that dynamically reads context, coordinates complex multi-query lookup strategies, routes sub-queries concurrently in parallel to avoid delay, and handles evolving multi-turn parameters.

Multiple Choice (Select Two)

Scenario 10: External App Tracing Architecture

You have a Microsoft Foundry project that contains a prompt agent used by a customer support web app. The agent is invoked from a Python service that does NOT run in the Foundry portal. You need to implement end-to-end tracing to capture latency breakdowns and exceptions across agent runs. Which two components can you use? (Each correct answer presents a complete solution).

A. a Log Analytics workspace

C. Application Insights

D. OpenTelemetry

E. the Azure Monitor Agent

Explanation: **OpenTelemetry** provides the cross-language open SDK standards used to instrument Python runtimes with semantic trace blocks. **Application Insights** acts as the cloud telemetry sink that visualizes and structures these traces and exceptions.

Dropdown SDK Integration

Scenario 11: Python Managed Identity Token Generation

You have a Python application named App1 that integrates with a Microsoft Foundry project named Project1. You need to ensure that App1 authenticates by using a Microsoft Entra managed identity and sends prompts to a deployed model by using the Azure OpenAI Responses API. How should you complete the code?

Code Location Target	Correct Code Selection
credential = [Dropdown_1]()	**DefaultAzureCredential**
response = openai_client.responses. [Dropdown_2](	**create**

Explanation: `DefaultAzureCredential` automatically extracts cloud token metadata fields from host-bound managed service contexts. The client-side method to submit parameters to the Responses collection endpoint is `.create()`.

Dropdown Model Execution Parameters

Scenario 12: Deterministic Consistency & Reasoning Quality

You have a Microsoft Foundry project containing a deployed chat model. You have a Python service that sends API requests to the model, integrated with an automated validation system comparing outputs against approved patterns. Wording differences cause validation mismatches. You need to update parameters to stabilize outputs and maximize reasoning quality using a model configured with an active thinking block.

Hyperparameter Field	Correct Target Value
temperature =	**0**
output_config = {"effort":	**"high"**

Explanation: Setting `temperature = 0` collapses probability variance, providing predictable string returns. For reasoning models, setting `"effort": "high"` triggers deepest structural thinking passes before rendering responses.

Multiple Choice

Scenario 13: Mixed Request Complexity Cost Optimization (Cascading)

You have a Microsoft Foundry project that serves a high-volume chat app. Most requests are simple FAQs, but some require advanced reasoning. You need to reduce costs and latency for common queries, without degrading the quality of the responses to complex questions. What should you do?

A. Increase the value of the max_tokens parameter for all the requests.

B. Route all the requests to a smaller model.

C. Use a model cascade that routes the requests to different models.

D. Route all the requests to the most capable model.

Explanation: A model cascade topology checks input complexity, resolving low-tier problems on cheap, fast nodes (like GPT-4o-mini), and escalating higher reasoning requirements to heavier models, conserving cluster budget.

Multiple Choice

Scenario 14: QR and Table Document Layout Extraction

You have a Microsoft Foundry project that contains an agent. The agent ingests scanned PDF vendor invoices that contain tables and embedded QR codes. The agent must preserve the PDF layout in the extracted output to ensure that downstream processing can reference sections and tables. You plan to call Azure Content Understanding in Foundry Tools. You need to extract content/layout elements and detect QR codes without requiring a language model deployment. Which built-in analyzer should you use?

A. prebuilt-read

B. prebuilt-layout

C. prebuilt-documentSearch

D. prebuilt-documentFieldSchema

Explanation: The `prebuilt-layout` analyzer isolates tables, bounding regions, hierarchy, text strings, and matrices like barcodes/QR tokens natively via computer vision networks without checking backend language transformers.

Multiple Choice

Scenario 15: Structure-Aware RAG Table Ingestion

You have a Microsoft Foundry project that contains an agent using a knowledge source built from documents in Azure Blob Storage (digitally scanned PDFs with multi-page tables). Your current ingestion job extracts only plain text, causing loss of table structure, headings, and page metadata. Users need to retrieve specific table rows across pages. You need to configure a RAG ingestion job that performs OCR, preserves hierarchies, and tracks page metadata. What should you do?

A. Use advanced data parsing to reingest the documents.

B. Use OCR and page-level chunking.

C. Use basic parsing and fixed-size chunking.

D. Use page-level OCR extraction and store each page as a single chunk.

Explanation: **Advanced data parsing** tracks structural grids and boundaries. It understands multi-page cell splits and constructs structured semantic objects, avoiding arbitrary mid-row character cuts common in simple chunking.

Multiple Choice

Scenario 16: Portal Markdown Ingestion Configuration

You have a Microsoft Foundry project named Project1 that contains an application that processes PDF vendor invoices. You need to configure Azure Document Intelligence in Foundry Tools to generate a Markdown output that preserves the sections and table structure of the PDFs. The solution must minimize development effort. What should you do?

A. Configure content=markdown when you analyze the document.

B. Configure output=figures when you analyze the PDF.

C. Increase the confidence threshold.

D. Set the output_content_format=ContentFormat.MARKDOWN value.

Explanation: Portal properties and string-based task forms in Foundry Tools accept the clean key-value pair `content=markdown` to instruct the underlying Document Intelligence pipeline to build markdown strings.

Multiple Choice

Scenario 17: Multi-Column Layout Extraction Sequence

You have a Microsoft Foundry project that contains an agent. You need to process mixed-format documents that contain scanned text, tables, and multicolumn layouts. The extracted content must preserve the document structure and be converted into the Markdown format for downstream reasoning. What should you configure first?

A. an Azure Language in Foundry Tools text analysis model deployment

B. a generative chat completion request

C. an Azure Content Understanding in Foundry Tools analyzer

D. an Azure OpenAI Responses API call that uses a multimodal model

Explanation: Content extraction and structural layout parsing represent the primary entry-point step. You must configure an **Azure Content Understanding analyzer** first to extract structural markdown layout blocks prior to executing model reasoning steps.

Multiple Choice

Scenario 18: Target-Locking Specific MCP Framework Tools

You have a Microsoft Foundry project that contains an agent. The agent has a Model Context Protocol (MCP) tool that queries a knowledge base stored in Azure AI Search. Some agent runs return answers from the base model without invoking the knowledge base, which results in responses without grounded citations. You need to add the correct tool_choice parameter to the code to deterministically force the agent to invoke the MCP tool on each run. What should you add?

A. tool_choice={"required"}

B. tool_choice={"auto"}

C. tool_choice={"type": "mcp"}

D. tool_choice={"type": "knowledge_base"}

Explanation: To narrow tool execution criteria to a specific catalog mechanism, pass a structured mapping object. Setting `{"type": "mcp"}` locks routing straight to the Model Context Protocol engine explicitly.

True / No Matrix

Scenario 19: Image Security Vector Evaluation (Case A)

You have a multimodal AI generative model that accepts image uploads and uses extracted image text to generate responses. You discover that users can upload unsafe images and embed hidden instructions into images to manipulate the model. You need to implement controls to mitigate the risk.

Proposed Solution: You configure a prompt shield for documents.

Does this meet the goal?

A. Yes

B. No

Explanation: Document prompt shields intercept text injection payloads inside parsed OCR files, but they lack computer vision matrix capabilities. Raw pixels displaying unsafe content will bypass this text filter completely.

True / No Matrix

Scenario 20: Image Security Vector Evaluation (Case B)

You have a multimodal AI generative model that accepts image uploads and uses extracted image text to generate responses. You discover that users can upload unsafe images and embed hidden instructions into images to manipulate the model. You need to implement controls to mitigate the risk.

Proposed Solution: You configure image moderation to block unsafe content before processing the images.

Does this meet the goal?

A. Yes

B. No

Explanation: Image moderation blocks inappropriate pixel layouts (violence, etc.). However, it does not scan text tokens for semantic injection payloads, leaving the model wide open to indirect instruction overrides.

True / No Matrix

Scenario 21: Image Security Vector Evaluation (Case C)

You have a multimodal AI generative model that accepts image uploads and uses extracted image text to generate responses. You discover that users can upload unsafe images and embed hidden instructions into images to manipulate the model. You need to implement controls to mitigate the risk.

Proposed Solution: You configure a prompt shield for user prompts.

Does this meet the goal?

A. Yes

B. No

Explanation: A user prompt shield isolates *direct* chat input text vectors (jailbreaks). It cannot parse image attachment streams or detect *indirect* injections embedded inside structural attachments.

True / No Matrix

Scenario 22: Regulatory Completeness (Case A - Reflection)

You have a Microsoft Foundry project that contains an agent. The agent generates summaries from retrieved policy documents. Users report that some responses omit required regulatory clauses, even when the clauses are present in the retrieved content. You need to improve response completeness.

Proposed Solution: You add a reflection pass that regenerates the response if the required clauses are missing.

Does this meet the goal?

A. Yes

B. No

Explanation: An active reflection loop validates drafts against absolute mandatory item schemas. If items are missing, it safely triggers internal loops to re-inject missing tokens, actively improving output accuracy.

True / No Matrix

Scenario 23: Regulatory Completeness (Case B - Max Tokens)

You have a Microsoft Foundry project that contains an agent. The agent generates summaries from retrieved policy documents. Users report that some responses omit required regulatory clauses, even when the clauses are present in the retrieved content. You need to improve response completeness.

Proposed Solution: You increase the value of the max_tokens parameter.

Does this meet the goal?

A. Yes

B. No

Explanation: Scaling up `max_tokens` extends the generation ceiling length but does not alter semantic instructions. It cannot compel a model to include specific items it chooses to omit.

True / No Matrix

Scenario 24: Regulatory Completeness (Case C - Blocker Evaluation)

You have a Microsoft Foundry project that contains an agent. The agent generates summaries from retrieved policy documents. Users report that some responses omit required regulatory clauses, even when the clauses are present in the retrieved content. You need to improve response completeness.

Proposed Solution: You run an evaluation flow that scores responses for completeness and blocks responses that fall below a defined threshold.

Does this meet the goal?

A. Yes

B. No

Explanation: Evaluation blockers drop ungrounded/incomplete outputs, hiding bad results from users. However, blocking alone does not fix or rewrite the generation step to actually yield a *complete* output.

True / No Matrix

Scenario 25: Regulatory Completeness (Case D - Temperature)

You have a Microsoft Foundry project that contains an agent. The agent generates summaries from retrieved policy documents. Users report that some responses omit required regulatory clauses, even when the clauses are present in the retrieved content. You need to improve response completeness.

Proposed Solution: You increase the value of the temperature parameter.

Does this meet the goal?

A. Yes

B. No

Explanation: Spiking `temperature` increases output token randomness and creativity. For rigid compliance workflows, high temperature worsens vocabulary drifting and leads to higher clause omission rates.